

07. live 系 node 構成ビルダ

solar_ws0129-main

2026-07-29

Contents

1 対象	3
2 このファイルを読む理由	3
3 呼び出し関係	3
3.1 どこから呼ばれるか	3
3.2 次に読むと理解が進むファイル	3
3.3 同一リポジトリ内の主要依存	3
4 ソース冒頭の把握	3
4.1 代表コード断片	3
5 主要構造の読み取り	4
5.1 主要クラス・関数	4
5.2 ファイルを上から読んだときの定義順	4
5.3 import 群の詳細	4
6 このファイルを読む前に必要な基礎知識	5
6.1 プログラム、プロセス、メモリ、オブジェクトを区別する	5
6.2 名前、代入、型、参照	5
6.3 関数、引数、戻り値、スコープ	6
6.4 module、package、import、console_scripts	6
6.5 例外、try/finally、with、資源解放	6
6.6 launch Action、Node Action、実行可能名、remapping	6
6.7 warm start は何を保存し、どう効くか	7
6.8 天候、route、補間、時刻、単位	7
6.9 freshness、filter、guard、fallback、fail-safe	7
6.10 制御、状態、入力、モデル予測制御 MPC	8
6.11 ROS graph、Node、topic、service、action、parameter	8
7 関数・クラスを上から順に解説	8
7.1 L11 関数 cfg_path	8
7.2 L15 関数 _drop_keys	9

7.3	L20 関数 <code>live_forecast_paths</code>	10
7.4	L39 関数 <code>build_live_nodes</code>	11
7.5	<code>launch</code> から起動するノード	13
8	処理の流れ	14
9	このファイルの位置づけ	14

1 対象

項目	内容
ファイル	mpc_solarcar/live_launch.py
ソース SHA-256	bee332a416c117f3f953081a7ba7f0c296ce0a3df527510ff7ee140059baceb7
種別	Python
区分	launch helper
役割	profile を読み、live と live_wifi で共通に使う node 群を Python から組み立てる。
起動文脈	launch/solar_race_live*.launch.py から呼ばれる。

2 このファイルを読む理由

profile を読み、live と live_wifi で共通に使う node 群を Python から組み立てる。

- forecast CSV の raw/corrected パスを用意する。
- mpc_node、logger、dashboard、preflight などの Node action を返す。
- use_wifi の有無で planner が読む forecast CSV を切り替える。

3 呼び出し関係

3.1 どこから呼ばれるか

- launch/solar_race_live.launch.py
- launch/solar_race_live_wifi.launch.py

3.2 次に読むと理解が進むファイル

- mpc_solarcar/solar_profile.py
- mpc_solarcar/mpc_node.py
- mpc_solarcar/speed_command_bridge_node.py

3.3 同一リポジトリ内の主要依存

- mpc_solarcar/solar_profile.py

4 ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

4.1 代表コード断片

```

if use_wifi and bool(wind_cfg.get("enabled", True)):
    mpc_forecast_csv = corrected_forecast_csv

nodes = [
    Node(
        package="mpc_solarcar",
        executable="solar_preflight_node",
        name="solar_preflight_node",
        parameters=[
            {
                "require_speed": True,
                "require_distance": True,
                "require_battery": True,
                "require_planner": True,
                **preflight_cfg,
            }
        ]
    )
]

```

5 主要構造の読み取り

主要関数は `cfg_path`, `live_forecast_paths`, `build_live_nodes`。 `launch` から起動する Node action は 9 件。

5.1 主要クラス・関数

行	種別	名前	補足
11	function	<code>cfg_path</code>	args: <code>profile_path</code> , <code>raw</code>
15	function	<code>_drop_keys</code>	args: <code>payload</code>
20	function	<code>live_forecast_paths</code>	args: <code>profile_path</code> , <code>cfg</code>
39	function	<code>build_live_nodes</code>	args: <code>profile_path</code> , <code>cfg</code>

5.2 ファイルを上から読んだときの定義順

行	内容
11	関数 <code>cfg_path</code> を定義する。
15	関数 <code>_drop_keys</code> を定義する。
20	関数 <code>live_forecast_paths</code> を定義する。
39	関数 <code>build_live_nodes</code> を定義する。

5.3 import 群の詳細

行	import 文	このファイルで読む理由
1	<code>from __future__ import annotations</code>	型ヒントの遅延評価を有効にし、自己参照型や forward reference を安全に扱うため。このファイル内での使用位置は少ないか、間接利用である。
3	<code>import shutil</code>	<code>shutil</code> モジュールを利用するため。このファイル内での主な使用位置は L35。
4	<code>from pathlib import Path</code>	ファイルやディレクトリを安全に扱うため。このファイル内での主な使用位置は L32, L33。

6	<code>from launch_ros. actions import Node</code>	ROS 2 ノード起動 action を launch から記述するため。 このファイル内での主な使用位置は L58, L72, L102, L113, L131, L140, L142, L145, ...。
8	<code>from solar_ profile import get_path, get_ section, resolve_relative_ path</code>	profile YAML 読込と検証から <code>get_path</code> , <code>get_section</code> , <code>resolve_relative_path</code> を読み込み、このファイルの 内部処理を分担させるため。実体ファイルは <code>mpc_solarcar/solar_profile.py</code> 。このファイル内での主 な使用位置は L12, L21, L22, L23, L24, L40, L41, L42, ...。

6 このファイルを読む前に必要な基礎知識

次の章は、構文や ROS 用語を既知と仮定しないための説明である。

6.1 プログラム、プロセス、メモリ、オブジェクトを区別する

ソースファイルはディスク上の文字列であり、それ自体は走っていない。Python 実行可能プログラムがソースを読み、OS がその実行を一つのプロセスとして管理する。プロセスには仮想メモリ、開いているファイル、スレッド、終了コードなどが対応する。

ソースファイル -> Python インタプリタが読み込む -> OS 上のプロセス -> Python オブジェクトをメモリに生成 -> 関数や callback を実行

「メモリ上に生成する」とは、実行中プロセスが使う記憶領域に、その値の型、属性、参照関係を表す実体を用意することである。変数は実体そのものというより、そのオブジェクトを指す名前として理解すると Python の代入が読みやすい。

```
node = MPCNode()
alias = node
# node と alias は同じオブジェクトを参照する。
```

一つのプロセスには複数スレッドを持たせられる。スレッドは同じプロセスのメモリを共有するため、受信 callback と最適化 callback が同じ `self.z` を書き換える場合は実行順と排他を検討する必要がある。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.2 名前、代入、型、参照

Python の代入文は、右辺を先に評価し、その結果のオブジェクトへ左辺の名前を結び付ける。‘`x = f()`’では、まず `f` を呼び、その戻り値が得られてから `x` が更新される。

‘`float(...)`’、‘`int(...)`’、‘`bool(...)`’は型名を呼び出して値を変換する。外部 CSV、YAML、ROS parameter から来た値は型が期待どおりとは限らないため、このリポジトリでは明示変換が多い。

‘`None`’は値が無いことを示す単一のオブジェクト、‘`math.nan`’は浮動小数点値ではあるが有効な数値ではないことを示す。両者は用途が異なるため、‘`is None`’と ‘`math.isfinite`’を使い分ける。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.3 関数、引数、戻り値、スコープ

‘def’は関数本体をその場で実行する文ではない。関数オブジェクトを作り、その名前を現在の名前空間へ登録する。‘f’は関数そのもの、‘f()’はその関数を呼んだ結果である。

仮引数は関数定義側の受け取り名、実引数は呼出側から渡す値である。位置引数、キーワード引数、既定値付き引数、‘*args’、‘**kwargs’は渡し方が異なる。

```
def clip_speed(value, minimum=0.0, maximum=110.0):  
    return min(maximum, max(minimum, value))  
  
v = clip_speed(120.0, maximum=90.0)
```

関数内で作った通常の名前はローカルスコープに属する。メソッドが‘self.z’を更新すればオブジェクトの状態が残るが、単に‘z’を更新しただけならその呼出しのローカル値である。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.4 module、package、import、console_scripts

Python ファイルは module として読み込める。複数 module をディレクトリにまとめたものが package である。‘from .model import SolarCarModel’の先頭の点は、現在と同じ package 内の model module を指す相対 import である。

import 時にはファイルのトップレベル文が上から一度実行される。‘def’や‘class’は関数・クラスを登録するが、その本体の通常処理は呼び出すまで走らない。

setuptools の ‘console_scripts’は、端末で使う実行可能名と ‘package.module:function’を対応付けるインストール時メタデータである。生成される実行用ラッパーは module を import し、指定関数を呼び、戻り値を終了コードとして扱う小さな入口である。

```
ROS launchのexecutable名 -> install済みconsole script -> mpc_solarcar.mpc_nodeをimport ->  
main()を呼ぶ
```

根拠資料

- [setuptools 公式: Entry Points / Console Scripts](#)
- [Python 公式チュートリアル: Classes](#)

6.5 例外、try/finally、with、資源解放

例外は通常の戻り値とは別経路で異常を呼出元へ伝える。‘try/except’は想定した異常を処理し、‘finally’は成功・失敗にかかわらず後始末を行う。

‘with open(...) as f:’は context manager を使い、ブロックを出るとファイルを閉じる。CSV やログの破損を避けるため、開いた資源の所有者と閉じる場所を明確にする。

‘except Exception: pass’はノードを止めない利点がある一方、入力異常を隠して原因追跡を難しくする。安全に関係する値では、少なくとも頻度制限付き警告、異常カウンタ、fallback 状態の publish を検討する。

6.6 launch Action、Node Action、実行可能名、remapping

‘launch_ros.actions.Node(...)’は rclpy の Node 基底クラスではなく、指定した package の executable をプロセスとして起動する launch Action である。

‘DeclareLaunchArgument’は launch 実行時に受け取る入力欄を宣言し、‘LaunchConfiguration’はその値を後で解決する substitution を表す。‘perform(context)’は実行時 context から確定文字列を取り出す。

launch の ‘name’は Node 名 override、‘parameters’は起動時 parameter、‘remappings’は Node や topic の既定名を別名へ対応付ける。launch は Python ファイルから main という名前を推測せず、executable としてインストールされた console script を起動する。

根拠資料

- [ROS 2 Humble 公式: Understanding nodes](#)
- [setuptools 公式: Entry Points / Console Scripts](#)

6.7 warm start は何を保存し、どう効くか

warm start は前回または offline 探索で得た入力系列を、次の最適化の初期候補として渡すことである。答えを固定するのではなく、探索開始点と候補 library の一員を与える。

$$u_i^{0,\text{new}} = \text{interp} \left(s_i^{\text{new}}; s_j^{\text{old}}, u_j^{*,\text{old}} \right)$$

距離基準計画では、現在より後ろの旧制御点を捨て、新しい絶対距離制御点へ補間して shift する。初期状態や天候が変われば cost 評価は新条件で行われるので、warm start が不適切でも他 seed、CEM、安全 fallback が補う設計が必要である。

warm start の効き方は、局所法なら収束先と反復数、CEM なら初期 mean または候補 pool、receding horizon なら前回解の時間・距離 shift として現れる。どの位置に渡しているかを呼出引数まで追う。

根拠資料

- [SciPy 公式: scipy.optimize.minimize](#)
- [Rubinstein and Kroese: The Cross-Entropy Method](#)

6.8 天候、route、補間、時刻、単位

予報は時刻だけの系列か、時刻と route 距離の 2 次元 grid になり得る。現在時刻 t と距離 s が grid 点の間なら、周囲の値から補間して日射、温度、風を求める。

UTC、現地 timezone、naive datetime を混ぜると数時間のずれが発生する。入力で timezone を確定し、内部比較は UTC、表示は現地時刻という役割分担が安全である。

route 距離の km、速度の km/h と m/s、時間の秒、energy の Wh と J を跨ぐため、変換係数 3.6、3600、1000 の意味を各式で確認する。

6.9 freshness、filter、guard、fallback、fail-safe

分散システムでは最後に受け取った値が現在も有効とは限らない。受信時刻と timeout から freshness を判定し、stale 値を計画状態へ無条件に同期しない。

filter は noise と一時的な飛び値を抑えるが、遅れを生む。slew limit は指令変化率を制限する。安全 guard は solver の cost 罰則とは別に、現在出力へ強制制約を適用する最後の防波堤である。

fallback は失敗時の代替動作を事前に決める設計である。前回計画保持、物理に基づく決定論的入力、停止、低速制限などから、故障 mode ごとに選ぶ。fallback 発生は status と log へ残し、正常解と区別する。

6.10 制御、状態、入力、モデル予測制御 MPC

制御対象の内部を表す状態を x 、操作入力を u 、外乱・予報を w とすると、離散モデルは ' $x[k+1] = f(x[k], u[k], w[k])$ ' と書ける。ソーラーカーでは SoC、電池温度、距離、速度などが状態候補、速度目標や駆動トルクが入力候補になる。

$$\min_{u_0, \dots, u_{N-1}} \sum_{k=0}^{N-1} \ell(x_k, u_k, w_k) + V_f(x_N)$$

MPC は現在状態から N ステップ先まで予測し、目的関数と制約を満たす入力系列を求める。ただし実際に適用するのは通常先頭入力だけで、次回は新しい実測状態から再び解く。これが **receding horizon** である。

予測モデル、目的関数、制約、ホライズン、solver、初期値のどれかが変わると答えも変わる。「MPC を使う」だけでは仕様は決まらず、これらを単位付きで追う必要がある。

根拠資料

- SciPy 公式: [scipy.optimize.minimize](#)

6.11 ROS graph、Node、topic、service、action、parameter

ROS graph は、実行中 Node と、それらが持つ publisher、subscription、service、action などの接続関係である。Python クラス、プロセス、ROS graph 上の Node 名は関連するが同一物ではない。

topic は継続データ向けの非同期一方向 publish/subscribe、service は短い request/response、action は時間のかかる目標へ feedback、cancel、result を持たせる。parameter は Node ごとの設定値である。

publisher が送った時点と subscription callback が実行される時点は同じとは限らない。通信遅延、QoS queue、executor 待ちを挟むため、センサ値には timestamp と freshness 判定が必要になる。

根拠資料

- ROS 2 Humble 公式: [Understanding nodes](#)
- ROS 2 Humble 公式: [Topics, Services, Actions](#)
- ROS 2 Humble 公式: [Parameters](#)

7 関数・クラスを上から順に解説

7.1 L11 関数 `cfg_path`

- 行範囲: L11-L12
- 所属: module 直下
- 定義: `cfg_path(profile_path, raw:str)->str`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `resolve_relative_path, str, strip`
- 戻り値の要点: `resolve_relative_path(profile_path.parent, raw) if str(raw or '') else ''`
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。

- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `resolve_relative_path(profile_path.parent, raw) if str(raw or "") else ""` を返す。

代表コード断片

```
def cfg_path(profile_path, raw: str) -> str:
    return resolve_relative_path(profile_path.parent, raw) if str(raw or "").strip() else ""
```

7.2 L15 関数 `_drop_keys`

- 行範囲: L15–L17
- 所属: module 直下
- 定義: `_drop_keys(payload:dict, *keys:str)->dict`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `items`, `set`
- 戻り値の要点: `{key: value for key, value in (payload or {}).items() if key not in blocked}`
- 主なローカル代入名: `blocked`, `key`, `value`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

上から順の処理

1. `blocked` に `set(keys)` の結果を代入する。
2. `{key: value for key, value in (payload or {}).items() if key not in blocked}` を返す。

代表コード断片

```
def _drop_keys(payload: dict, *keys: str) -> dict:
    blocked = set(keys)
    return {key: value for key, value in (payload or {}).items() if key not in blocked}
```

7.3 L20 関数 live_forecast_paths

- 行範囲: L20–L36
- 所属: module 直下
- 定義: `live_forecast_paths(profile_path, cfg)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Path`, `cfg_path`, `copy2`, `exists`, `get`, `get_path`, `get_section`, `mkdir`, `str`
- 戻り値の要点: (`base_path`, `raw_path`, `corrected_path`)
- 主なローカル代入名: `base_path`, `corrected_path`, `live_cfg`, `raw_path`, `runtime_path`, `target`, `weather_cfg`, `wind_cfg`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 3、ループ 1、`try` 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. `live_cfg` に `get_section(cfg, 'live')` の結果を代入する。
2. `weather_cfg` に `get_section(live_cfg, 'weather')` の結果を代入する。
3. `wind_cfg` に `get_section(live_cfg, 'wind_model')` の結果を代入する。
4. `base_path` に `get_path(cfg, profile_path, 'forecast_csv')` の結果を代入する。
5. `raw_path` に `cfg_path(profile_path, str(weather_cfg.get('raw_forecast_csv', '')))` の結果を代入する。
6. 条件 `not raw_path` を判定し、真なら内部処理を行う。
7. `raw_path` に `cfg_path(profile_path, 'outputs/runtime/live_forecast_raw.csv')` の結果を代入する。
8. `corrected_path` に `cfg_path(profile_path, str(wind_cfg.get('corrected_forecast_csv', '')))` の結果を代入する。
9. 条件 `not corrected_path` を判定し、真なら内部処理を行う。
10. `corrected_path` に `cfg_path(profile_path, 'outputs/runtime/live_forecast_corrected.csv')` の結果を代入する。
11. (`raw_path`, `corrected_path`) を順に走査し、各要素を `runtime_path` に入れて処理する。
12. `target` に `Path(runtime_path)` の結果を代入する。
13. 条件 `not target.exists()` and `Path(base_path).exists()` を判定し、真なら内部処理を行う。
14. `target.parent.mkdir(...)` を実行する。
15. `shutil.copy2(...)` を実行する。
16. (`base_path`, `raw_path`, `corrected_path`) を返す。

代表コード断片

```
def live_forecast_paths(profile_path, cfg):
    live_cfg = get_section(cfg, "live")
    weather_cfg = get_section(live_cfg, "weather")
    wind_cfg = get_section(live_cfg, "wind_model")
    base_path = get_path(cfg, profile_path, "forecast_csv")
    raw_path = cfg_path(profile_path, str(weather_cfg.get("raw_forecast_csv", "")))
    if not raw_path:
        raw_path = cfg_path(profile_path, "outputs/runtime/live_forecast_raw.csv")
    corrected_path = cfg_path(profile_path, str(wind_cfg.get("corrected_forecast_csv", "")))
    if not corrected_path:
        corrected_path = cfg_path(profile_path, "outputs/runtime/live_forecast_corrected.csv")
    for runtime_path in (raw_path, corrected_path):
        target = Path(runtime_path)
        if not target.exists() and Path(base_path).exists():
            target.parent.mkdir(parents=True, exist_ok=True)
            shutil.copy2(base_path, target)
    return base_path, raw_path, corrected_path
```

7.4 L39 関数 build_live_nodes

- 行範囲: L39–L175
- 所属: module 直下
- 定義: `build_live_nodes(profile_path, cfg, *, use_wifi: bool)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Node`, `_drop_keys`, `append`, `bool`, `cfg_path`, `float`, `get`, `get_path`, `get_section`, `int`, `live_forecast_paths`, `str`
- 戻り値の要点: `nodes`
- 主なローカル代入名: `autocal_cfg`, `base_forecast_csv`, `command_cfg`, `corrected_forecast_csv`, `distance_cfg`, `grade_cfg`, `live_cfg`, `live_logging_cfg`, `logging_cfg`, `mpc_forecast_csv`, `nodes`, `preflight_cfg`, `raw_forecast_csv`, `runtime_cfg`, `weather_cfg`, `wind_cfg`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 6、ループ 0、`try` 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. `runtime_cfg` に `get_section(cfg, 'runtime')` の結果を代入する。
2. `logging_cfg` に `get_section(cfg, 'logging')` の結果を代入する。
3. `live_cfg` に `get_section(cfg, 'live')` の結果を代入する。
4. `weather_cfg` に `get_section(live_cfg, 'weather')` の結果を代入する。

5. autocal_cfg に get_section(live_cfg, 'autocal') の結果を代入する。
6. command_cfg に get_section(live_cfg, 'command_bridge') の結果を代入する。
7. distance_cfg に get_section(live_cfg, 'distance') の結果を代入する。
8. grade_cfg に get_section(live_cfg, 'grade') の結果を代入する。
9. wind_cfg に get_section(live_cfg, 'wind_model') の結果を代入する。
10. live_logging_cfg に get_section(live_cfg, 'logging') の結果を代入する。
11. preflight_cfg に get_section(live_cfg, 'preflight') の結果を代入する。
12. (base_forecast_csv, raw_forecast_csv, corrected_forecast_csv) に live_forecast_paths(profile_path, cfg) の結果を代入する。
13. mpc_forecast_csv に raw_forecast_csv の結果を代入する。
14. 条件 use_wifi and bool(wind_cfg.get('enabled', True)) を判定し、真なら内部処理を行う。
15. mpc_forecast_csv に corrected_forecast_csv の結果を代入する。
16. nodes に [Node(package='mpc_solarcar', executable='solar_preflight_node', name='solar_preflight_node', parameters=[{'require_speed': True, 'require_distance': True, 'require_battery': True, 'require_planner': True, **preflight_cfg}]), Node(package='mpc_solarcar', executable='mpc_node', name='mpc_node', parameters=[{'forecast_csv': mpc_forecast_csv, 'route_profile_csv': get_path(cfg, profile_path, 'route_profile_csv'), 'speed_profile_csv': get_path(cfg, profile_path, 'speed_profile_csv'), 'stop_yaml': get_path(cfg, profile_path, 'stop_yaml'), 'drive_schedule_yaml': get_path(cfg, profile_path, 'drive_schedule_yaml'), 'initial_upper_policy_csv': get_path(cfg, profile_path, 'initial_upper_policy_csv'), 'drive_eff_map': get_path(cfg, profile_path, 'drive_eff_map'), 'regen_eff_map': get_path(cfg, profile_path, 'regen_eff_map'), 'rint_map': get_path(cfg, profile_path, 'rint_map'), 'panel_eff_map': get_path(cfg, profile_path, 'panel_eff_map'), 'mppt_eff_map': get_path(cfg, profile_path, 'mppt_eff_map'), 'drive_map_eco': get_path(cfg, profile_path, 'drive_map_eco'), 'drive_map_power': get_path(cfg, profile_path, 'drive_map_power'), 'regen_map_eco': get_path(cfg, profile_path, 'regen_map_eco'), 'regen_map_power': get_path(cfg, profile_path, 'regen_map_power'), 'ocv_soc_map': get_path(cfg, profile_path, 'ocv_soc_map'), 'params_yaml': str(profile_path), 'profile_runtime_mode': 'live', 'forecast_time_mode': str(live_cfg.get('forecast_time_mode', runtime_cfg.get('forecast_time_mode', 'absolute'))), 'forecast_time_tz': str(live_cfg.get('forecast_time_tz', runtime_cfg.get('forecast_time_tz', 'Australia/Darwin'))), 'forecast_start_time_utc': str(get_section(cfg, 'simulation').get('forecast_start_time_utc', get_section(cfg, 'simulation').get('start_utc', '')))}], Node(package='mpc_solarcar', executable='dashboard_node', name='dashboard_node', parameters=[{'host': str(runtime_cfg.get('dashboard_host', '0.0.0.0')), 'port': int(runtime_cfg.get('dashboard_port', 8080))}], Node(package='mpc_solarcar', executable='solar_logger_node', name='solar_logger_node', parameters=[{'log_dir': cfg_path(profile_path, str(logging_cfg.get('log_dir', 'outputs/logs'))), 'file_prefix': str(live_logging_cfg.get('file_prefix', 'solar_live')), 'log_rate_hz': float(logging_cfg.get('log_rate_hz', 2.0)), 'output_speed_topic': str(command_cfg.get('output_vehicle_speed_cmd_kmh')), 'output_drive_mode_topic': str(command_cfg.get('output_drive_mode_topic', 'vehicle/drive_mode_cmd'))}]))] の結果を代入する。
17. 条件 bool(command_cfg.get('enabled', True)) を判定し、真なら内部処理を行う。
18. nodes.append(...) を実行する。
19. 条件 bool(live_cfg.get('use_distance_node', True)) を判定し、真なら内部処理を行う。
20. nodes.append(...) を実行する。
21. 条件 bool(live_cfg.get('use_grade_node', True)) を判定し、真なら内部処理を行う。
22. nodes.append(...) を実行する。
23. 条件 bool(weather_cfg.get('enabled', True)) を判定し、真なら内部処理を行う。

24. nodes.append(...) を実行する。
25. 条件 bool(autocal_cfg.get('enabled', True)) を判定し、真なら内部処理を行う。
26. nodes.append(...) を実行する。
27. nodes を返す。

代表コード断片

```
def build_live_nodes(profile_path, cfg, *, use_wifi: bool):
    runtime_cfg = get_section(cfg, "runtime")
    logging_cfg = get_section(cfg, "logging")
    live_cfg = get_section(cfg, "live")
    weather_cfg = get_section(live_cfg, "weather")
    autocal_cfg = get_section(live_cfg, "autocal")
    command_cfg = get_section(live_cfg, "command_bridge")
    distance_cfg = get_section(live_cfg, "distance")
    grade_cfg = get_section(live_cfg, "grade")
    wind_cfg = get_section(live_cfg, "wind_model")
    live_logging_cfg = get_section(live_cfg, "logging")
    preflight_cfg = get_section(live_cfg, "preflight")

    base_forecast_csv, raw_forecast_csv, corrected_forecast_csv = live_forecast_paths(
        profile_path, cfg)
    mpc_forecast_csv = raw_forecast_csv
    if use_wifi and bool(wind_cfg.get("enabled", True)):
        mpc_forecast_csv = corrected_forecast_csv

    nodes = [
        Node(
            package="mpc_solarcar",
            executable="solar_preflight_node",
            name="solar_preflight_node",
            parameters=[
                {
                    "require_speed": True,
                    "require_distance": True,
                    "require_battery": True,
                    "require_planner": True,
                    **preflight_cfg,
                }
            ],
        ),
        Node(
            package="mpc_solarcar",
            ...
```

7.5 launch から起動するノード

行	executable	package / name
58	solar_preflight_node	package=mpc_solarcar, name=solar_preflight_node
72	mpc_node	package=mpc_solarcar, name=mpc_node
102	dashboard_node	package=mpc_solarcar, name=dashboard_node
113	solar_logger_node	package=mpc_solarcar, name=solar_logger_node
131	speed_command_bridge_node	package=mpc_solarcar, name=speed_command_bridge_node
140	distance_node	package=mpc_solarcar, name=distance_node
142	grade_node	package=mpc_solarcar, name=grade_node

145	weather_fetch_node	package=mpc_solarcar, name=weather_fetch_node
168	solar_autocal_node	package=mpc_solarcar, name=solar_autocal_node

8 処理の流れ

1. ソース中の主要関数を通じて処理を進める。

9 このファイルの位置づけ

この資料群では、`mpc_solarcar/live_launch.py` を **launch helper** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。